

第7章：Policy Gradient (PG, 策略梯度)：从策略梯度定理到 REINFORCE with Baseline

7.1 本章符号说明

符号/缩写	English	中文	含义
PG	Policy Gradient	策略梯度	直接对策略参数求梯度、最大化期望回报的方法。
REINFORCE	REINFORCE Algorithm	REINFORCE 算法	使用完整轨迹和 Monte Carlo return 估计 Policy Gradient 的基础算法。
MC	Monte Carlo	蒙特卡洛	等待未来奖励被实际采样出来，用样本回报估计期望。
$S_t / A_t / R_{t+l}$	State / Action / Reward Random Variable	状态 / 动作 / 奖励随机变量	时刻 t 的状态、动作，以及执行动作后收到的奖励。小写 s_t, a_t, r_{t+l} 表示一次采样值。
T	Episode Horizon / Terminal Time	轨迹终止时刻	一条 episode 的最后时刻。
$\pi_\theta(a s)$	Parameterized Policy	参数化策略	参数为 θ 的动作条件分布；给定状态 s 后为动作 a 分配概率或概率密度。
θ	Policy Parameters	策略参数	Policy network 中需要学习的参数。
τ	Trajectory	轨迹	一段 state-action-reward 序列。
$p_\theta(\tau)$	Trajectory Probability / Density	轨迹概率 / 密度	当前策略产生整条轨迹 τ 的概率或密度。
$P(s' s,a)$	Transition Probability	状态转移概率	环境从 (s,a) 转移到下一状态 s' 的概率，默认不依赖 θ 。
γ	Discount Factor	折扣因子	$0 \leq \gamma \leq 1$ ，控制较远未来奖励的权重。
G_t	Return / Reward-to-go	从 t 开始的回报	从时刻 t 之后实际采样到的累计折扣奖励。
$J(\theta)$	Policy Objective	策略目标函数	当前策略的期望回报，是策略需要最大化的对象。
$Q_\pi(s,a)$	Action-value Function	动作价值函数	固定 $S_t=s, A_t=a$ 后，对未来随机回报 G_t 取条件期望。
$V_\pi(s)$	State-value Function	状态价值函数	固定 $S_t=s$ 后，按当前策略选择动作时 G_t 的条件期望。
$b_\phi(s)$	Learned Baseline	可学习基线	参数为 ϕ 的状态参考值，用于降低 Policy Gradient 方差。
$A_\pi(s,a)$	Advantage Function	优势函数	$Q_\pi(s,a) - V_\pi(s)$ ，表示动作 a 比状态 s 下当前策略的平均动作好多少。

符号/缩写	English	中文	含义
$\hat{A}_t^{MC}$	Monte Carlo Advantage Estimate	蒙特卡洛优势估计	用单条轨迹的 $G_t - b_\phi(s_t)$ 估计理论 advantage；帽子表示它是估计量。
$f_\theta(s)$	Policy Logits	策略 logits	离散策略网络在 softmax 之前输出的未归一化分数。
$z_i / Z(s)$	Action Logit / Softmax Normalizer	动作 logit / Softmax 归一化因子	z_i 是动作 i 的未归一化分数； $Z(s)$ 是所有动作指数分数之和。
$I[\cdot]$	Indicator Function	指示函数	方括号内条件成立时取 1，否则取 0。
c_t	Score Function Gradient Component	得分函数梯度分量	已采样动作的 log probability 对指定 logit 的偏导数。
$\mu_\theta(s) / \sigma_\theta(s)$	Gaussian Mean / Standard Deviation	高斯均值 / 标准差	连续策略为动作分布输出的参数。
$N(x; \mu, \sigma)$	Gaussian Probability Density	高斯概率密度	均值为 μ 、标准差为 σ 的高斯分布在 x 处的密度；分号左边是取值，右边是分布参数。
K_t / U_t	Discrete Type / Continuous Parameter Random Variable	离散类型 / 连续参数随机变量	混合动作 $A_t = (K_t, U_t)$ 的两个组成部分。
θ_d / θ_c	Discrete-head / Continuous-head Parameters	离散分支 / 连续分支参数	混合策略中 categorical head 和 conditional continuous head 的参数。
$g_{MC}(\tau)$	Monte Carlo Policy-gradient Estimate	蒙特卡洛策略梯度估计	一条采样轨迹 τ 构造出的样本梯度。
$g_{base}(\tau)$	Baseline-adjusted Gradient Estimate	带基线的梯度估计	将 g_{MC} 中的样本权重 G_t 换成 $G_t - b_\phi(s_t)$ 。
L_{surr}	Surrogate Policy Loss	代理策略损失	为了使用自动微分而构造的标量函数；它的负梯度等于希望使用的样本梯度上升方向。
$L_{policy} / L_{baseline}$	Policy / Baseline Loss	策略 / 基线损失	分别更新策略参数 θ 和基线参数 ϕ 。
L	Loss Function	损失函数	实现中使用梯度下降最小化的目标。
α_π / α_V	Policy / Baseline Learning Rate	策略 / 基线学习率	分别控制 policy 和 learned baseline 的更新步长。

7.2 问题定义与算法路线：如何直接优化随机策略

7.2.1 从动作价值函数转向动作概率分布

前面的 value-based 方法先学习动作价值 $Q(s, a)$ ，再通过 $\arg \max_a Q(s, a)$ 选择动作。本章改为直接学习动作分布 $\pi_\theta(a|s)$ 。

真正需要解决的问题只有一个：

动作由策略随机采样，奖励由不可微或未知的环境产生，怎样求策略参数 θ 对期望回报 $J(\theta)$ 的梯度？

Policy Gradient (PG, 策略梯度) 的答案不是穿过环境反向传播，而是调整产生已采样动作和轨迹的概率。

7.2.2 本章要推导的两个算法

本章全部公式只服务于两个可执行算法：**REINFORCE** 使用完整回报 G_t ；**REINFORCE with Baseline** 使用相对回报 $G_t - b_\phi(S_t)$ 。推导顺序如下。

理论起点：Policy Gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t | S_t) Q_\pi(S_t, A_t) \right]$$

它说：如果某个动作的长期价值较高，就沿着提高该动作 log probability 的方向更新。

算法一：REINFORCE 用实际回报构造样本梯度估计

$$g_{MC}(\tau) = \sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t | S_t) G_t$$

G_t 是一条真实轨迹提供的 Monte Carlo (MC, 蒙特卡洛) 样本，因此这个梯度估计无 bootstrap 偏差，但方差较高。

REINFORCE 的工程实现：代理 Loss 让自动微分产生同一个样本梯度

$$L_{surv}(\theta; \tau) = - \sum_{t=0}^{T-1} \gamma^t \log \pi_\theta(A_t | S_t) G_t$$

$$\nabla_\theta L_{surv} = -g_{MC}(\tau)$$

所以对 g_{MC} 做梯度上升"和"对 L_{surv} 做梯度下降"是同一次参数更新。代理 loss 没有引入新的学习信号，里面仍然是同一个 G_t 。

算法二：REINFORCE with Baseline 只替换样本权重

$$g_{base}(\tau) = \sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t | S_t) [G_t - b_\phi(S_t)]$$

只要 $b_\phi(S_t)$ 在给定状态后不依赖当前动作，它就不改变梯度期望；好的 baseline 会让有限样本梯度更稳定。当 $b_\phi(S_t)$ 近似 $V_\pi(S_t)$ 时， $G_t - b_\phi(S_t)$ 是理论 Advantage (优势) 的 Monte Carlo 估计。

7.2.3 从期望回报到 REINFORCE with Baseline 的推导路线

期望回报 $J(\theta)$

- > Log-derivative Trick: 把“对分布求导”变成 log-probability 梯度
- > Reward-to-go: 只让动作承担其后的奖励
- > REINFORCE: 用一条轨迹的 G_t 构造样本梯度估计
- > Surrogate Loss: 把同一个样本梯度包装成自动微分可最小化的标量
- > Baseline: 用 $G_t - b_\phi(s_t)$ 降低样本梯度方差

其中从样本梯度到代理 loss 只是实现形式转换，不是算法思想再次发生变化。

Categorical、Gaussian 和混合动作策略不是三套不同算法。它们只是计算 $\log \pi_\theta(a_t | s_t)$ 的方式不同，统一放到主推导之后的 7.9 节。

7.2.4 与 Actor-Critic 的分界：是否使用 Bootstrap

本章始终等待完整未来奖励：

- REINFORCE 使用完整 G_t 。
- REINFORCE with baseline 使用 $G_t - b_\phi(S_t)$ 。
- Baseline 虽然可以是神经网络，但监督标签仍是完整 G_t 。

第8章才让价值网络使用 Bootstrap（自举）提前构造 target，并引入 Temporal Difference (TD, 时序差分)、n-step 和 Generalized Advantage Estimation (GAE, 广义优势估计)。

因此 learned baseline 是否算 Critic 需要区分宽泛定义与课程中的算法命名，7.6.6 会专门给出结论。

7.3 第一步：用轨迹期望定义 Policy Objective（策略目标）

7.3.1 Trajectory（轨迹）与 Reward-to-go（未来回报）

一条完整轨迹写成：

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$$

从时刻 t 开始的 reward-to-go 为：

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$$

G_t 是随机变量，因为未来状态转移和未来动作都可能随机。

7.3.2 用期望回报定义 $J(\theta)$

对从初始状态开始的 episodic task，策略目标可写成：

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[G_0]$$

它表示：按当前策略反复采样轨迹，期望上希望获得更大的 G_0 。

7.3.3 为什么不能穿过环境直接反向传播

目标中的轨迹分布 $p_\theta(\tau)$ 随策略参数 θ 变化：策略改变动作概率，动作又改变后续状态与奖励。

不能把环境当作普通监督学习网络直接反传，原因包括：

- 离散动作采样不可微。
- 环境转移未必可微，也未必知道其内部函数。
- 奖励可能延迟很多步才出现。

Policy Gradient 的关键不是穿过环境反传，而是对“产生轨迹的概率”求导。

7.4 第二步：推导 Policy Gradient（策略梯度）

7.4.1 Log-derivative Trick：把分布导数改写成期望

对任意可微概率分布，Log-derivative Trick（对数导数技巧，也叫 Score Function Trick）为：

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

将它用到轨迹期望：

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G_0 d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) G_0 d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G_0 d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) G_0] \end{aligned}$$

这一步把“对随 θ 变化的轨迹分布求导”转成了“在当前分布下采样，再对采样轨迹的 log probability 求导”。

7.4.2 展开轨迹概率：环境项为什么不要求导

轨迹概率分解为：

$$p_{\theta}(\tau) = p(S_0) \prod_{t=0}^{T-1} \pi_{\theta}(A_t | S_t) P(S_{t+1} | S_t, A_t)$$

取对数后乘积变成求和。初始状态分布 $p(S_0)$ 和环境转移 P 默认不依赖策略参数，因此：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

代回目标梯度：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_0]$$

环境决定哪些轨迹和奖励会被采样，所以环境当然影响梯度样本；“环境项消失”只表示不需要对环境转移函数求导。

7.4.3 Reward-to-go：只让动作承担之后的奖励

上式让每个时间步的动作梯度都乘上整条轨迹回报 G_0 ，但 A_t 无法影响时刻 t 之前已经发生的奖励。

从期望梯度中去掉这些过去奖励项，不会改变梯度期望，但可以减少噪声。对从 G_0 定义的折扣目标，严格写法为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t]$$

外层 γ^t 来自 G_0 中第 t 时刻奖励的折扣位置，而 G_t 内部从当前时刻重新以 γ^0 开始计算。

很多教材和实现在以下情况不显式写外层 γ^t ：

- episodic 任务使用 $\gamma=1$ 。
- 将时间权重吸收进采样分布或目标定义。
- 工程上直接使用 G_t 作为样本权重。

面试时要能说清所采用的目标定义，不要在不同公式约定之间机械比较是否“少了一个 γ^t ”。

7.4.4 Iterated Expectation：从 G_t 得到 $Q_{\pi}(s,a)$

即使当前 (S_t, A_t) 完全相同，不同的环境转移和后续动作也会产生不同 G_t 。固定 $S_t=s, A_t=a$ 后对这些未来随机性取平均，定义出：

$$Q_{\pi}(s,a) = \mathbb{E}_{\pi} [G_t | S_t=s, A_t=a]$$

它不是突然换了一个学习信号，而是给“ G_t 的条件均值”起名为 Action-value Function。

对任意一个时间步，根据 iterated expectation (迭代期望，也叫全期望公式)：

$$\begin{aligned} & \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t] \\ &= \mathbb{E}_{S_t, A_t} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \mathbb{E}[G_t | S_t, A_t]] \\ &= \mathbb{E}_{S_t, A_t} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) Q_{\pi}(S_t, A_t)] \end{aligned}$$

已知 (S_t, A_t) 后， $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 已是确定量，因此可以移出内层条件期望。保留与上一小节一致的时间权重，Policy Gradient 可写成：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [\sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) Q_{\pi}(S_t, A_t)]$$

这一推导给出两个层次：

层次	数学对象	含义
单条轨迹	G_t	本次从 (s_t, a_t) 之后实际观测到的随机回报。
总体期望	$Q_{\pi}(s, a)$	固定 (s, a) 后，所有可能 G_t 的条件平均。

Policy Gradient 的期望表达式中出现 Q_{π} 并不表示必须训练 Q network。下一节会用实际轨迹中的 G_t 估计这个期望梯度。

7.5 第三步：REINFORCE 用完整轨迹估计策略梯度

7.5.1 用实际回报 G_t 估计未知的 $Q_{\pi}(s, a)$

REINFORCE 不先学习 Q_{π} 。它采样完整轨迹，用本次观测到的 G_t 作为 $Q_{\pi}(s_t, a_t)$ 的 Monte Carlo 样本。

保留严格的折扣时间权重，一条轨迹给出的梯度估计为：

$$g_{MC}(\tau) = \sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

使用梯度上升更新策略：

$$\theta \leftarrow \theta + \alpha_{\pi} g_{MC}(\tau)$$

到这里只得到了数学上希望沿着前进的向量 $g_{MC}(\tau)$ 。如果手工执行梯度上升，这已经足够；代理 loss 只是把它接到常见自动微分接口上。

7.5.2 Surrogate Loss：让自动微分执行同一次更新

先给结论，避免把它们误认为两个算法步骤：

写法	更新方式	角色
$g_{MC}(\tau)$	$\theta \leftarrow \theta + \alpha_{\pi} g_{MC}$	数学上估计出的梯度上升方向。
$L_{surrogate}(\theta; \tau)$	$\theta \leftarrow \theta - \alpha_{\pi} \nabla_{\theta} L_{surrogate}$	自动微分所需的标量包装。

只要 $\nabla_{\theta} L_{surrogate} = -g_{MC}$ ，两种写法得到完全相同的参数更新。自动微分框架的常规接口是：给定一个标量 loss，计算它的梯度，再用梯度下降最小化它。

因此对已经采样完成的轨迹 τ ，构造 Surrogate Loss（代理损失）：

$$L_{surr}(\theta; \tau) = - \sum_{t=0}^{T-1} \gamma^t \log \pi_{\theta}(a_t | s_t) G_t$$

对这个标量求梯度。求导时，轨迹中已采样的 s_t, a_t, G_t 都当作已知数值，只对 $\log \pi_{\theta}$ 中的策略参数 θ 求导：

$$\begin{aligned} \nabla_{\theta} L_{surr}(\theta; \tau) &= - \sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \\ &= -g_{MC}(\tau) \end{aligned}$$

框架使用梯度下降：

$$\begin{aligned} \theta &\leftarrow \theta - \alpha_{\pi} \nabla_{\theta} L_{surr} \\ &= \theta - \alpha_{\pi} [-g_{MC}(\tau)] \\ &= \theta + \alpha_{\pi} g_{MC}(\tau) \end{aligned}$$

所以三个对象有层次区别，但只有一个最终目的：提高期望回报。

对象	是什么	关系
$J(\theta)$	真正希望最大化的期望回报	原始优化目标。
$g_{MC}(\tau)$	一条轨迹给出的 $\nabla_{\theta} J$ 样本估计	希望沿着它做梯度上升。
$L_{surr}(\theta; \tau)$	为了调用自动微分而构造的代理损失	$\nabla_{\theta} L_{surr} = -g_{MC}$ 。

L_{surr} 的数值本身不是 $-J(\theta)$ 的 Monte Carlo 估计，也不一定适合跨 batch 比较“越小策略越好”。它是为当前 on-policy 样本构造的局部代理目标，关键是它对 θ 的梯度正确。

看一个单时间步就很直观：

$$L_t = -G_t \log \pi_{\theta}(a_t | s_t)$$

为了只看更新方向，暂记：

$$x_t = \log \pi_{\theta}(a_t | s_t)$$

把已采样的 G_t 当作常数，则：

$$\partial L_t / \partial x_t = -G_t$$

梯度下降更新为：

$$\begin{aligned} x_t &\leftarrow x_t - \alpha_{\pi} \partial L_t / \partial x_t \\ &= x_t + \alpha_{\pi} G_t \end{aligned}$$

- $G_t > 0$: x_t 增大；由于 $\log$ 单调递增，该动作的概率或密度提高。
- $G_t < 0$: x_t 减小，该动作的概率或密度降低。
- $G_t = 0$: 这个时间步不提供策略更新信号。

G_t 在这个 loss 中是已采样的权重，实现中应当 `detach` 或 `stop_gradient`。对策略依赖采样分布的影响，已经由前面推导的 score-function gradient 处理；不再通过环境或 G_t 反向传播。

7.5.3 REINFORCE 的完整训练流程

```

initialize policy pi_theta
repeat:
    use current pi_theta to sample complete episodes
    for each episode:
        compute every reward-to-go G_t backward
        read the saved log pi_theta(a_t | s_t)
        accumulate -gamma^t * log pi_theta(a_t | s_t) * G_t
    average the loss over sampled episodes
    update theta with gradient descent

```

一次训练样本的数据流为：

```

s_t
-> policy distribution pi_theta(.|s_t)
-> sample a_t and save log_prob_t
-> environment returns r_{t+1}, s_{t+1}
-> after episode ends, compute G_t
-> loss_t = -gamma^t * log_prob_t * G_t

```

7.5.4 REINFORCE 属于哪类强化学习算法

REINFORCE 属于：

- **Policy-based**：直接更新动作分布。
- **On-policy**：训练数据由当前正在更新的策略产生。
- **Monte Carlo**：需要观测完整未来奖励才能得到 G_t 。
- **Model-free**：不需要已知环境的转移函数。
- **No bootstrap**：不用当前价值估计去构造 G_t 。

7.5.5 为什么还要改进：Monte Carlo 梯度方差高

固定同一 (s, a) ，后续环境随机性和后续动作采样仍可能产生差异很大的 G_t 。

例如五次从同一 (s, a) 开始得到：

```
G_t = 2, 18, -4, 25, 9
```

样本均值会逐渐接近 $Q_\pi(s, a)$ ，但单次更新可能被偶然的 25 或 -4 强烈影响。下一节会看到：baseline 的数学作用是降低这种样本方差；在单次更新上，它表现为用“相对状态平均水平”判断动作应被鼓励还是抑制。

7.6 第四步：REINFORCE with Baseline 用相对回报降低方差

这一节不再引入第二套 Policy Gradient。它只把上一节样本梯度和代理 loss 中的权重 G_t ，统一替换为 $G_t - b(S_t)$ ：

原始 REINFORCE	加入 Baseline 后
样本梯度使用 G_t	样本梯度使用 $G_t - b(S_t)$
代理 loss 使用 $-G_t \log \pi$	代理 loss 使用 $-(G_t - b(S_t)) \log \pi$

7.6.1 Baseline 的动机：比较同一状态下的相对表现

原始 REINFORCE 用 G_t 作为动作的样本权重。但一个回报的绝对数值未必能反映动作相对该状态的平均水平是好还是坏。

例如某个极难状态的平均回报是 -100 ，某动作这次得到 -60 。虽然 -60 仍为负，它其实比该状态的平均表现好 40 。

因此将学习信号改为：

$$G_t - b(S_t)$$

baseline 把“绝对回报”变成“相对参考水平的超额回报”。

7.6.2 数值例子：为什么均值不变而单次更新更可靠

固定当前策略参数后，真实 Policy Gradient 是对所有可能轨迹求平均得到的方向；训练时只能采样有限轨迹。记 g_t 为时刻 t 的 Sample Policy Gradient（样本策略梯度）：

$$g_t = G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

加入 baseline 后，样本梯度变为：

$$g_t^b = [G_t - b(S_t)] \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

下一小节将证明：只要 $b(S_t)$ 不依赖当前动作，两者的大量样本平均值相同；但单条样本的更新可以完全不同。下面用两个等概率动作逐步计算。

7.6.2.1 Softmax Score Function Gradient 从哪里来

设较好动作的 logit（Softmax 前的未归一化分数）为 z_{good} 。只观察样本梯度在这个参数上的分量：

$$c_t := \partial_{z_{good}} \log \pi(A_t | s)$$

$\partial_{z_{good}}$ 表示“对较好动作的 logit z_{good} 求偏导”。 c_t 是 Score Function Gradient（得分函数梯度）的一个分量。这里的 score 是概率统计术语，不是奖励；它表示参数变化时，已采样动作的 log probability 如何变化。

下面把原来省略的 Softmax 求导展开。定义 $Z(s)$ 为 Softmax Normalizer（Softmax 归一化因子）：

$$Z(s) = \exp(z_{good}) + \exp(z_{bad})$$

两个动作的概率可以写成：

$$\begin{aligned} \pi(good | s) &= \exp(z_{good}) Z(s)^{-1} \\ \pi(bad | s) &= \exp(z_{bad}) Z(s)^{-1} \end{aligned}$$

对于本次实际采样到的动作 A_t ：

$$\log \pi(A_t | s) = z_{A_t} - \log Z(s)$$

对 z_{good} 求偏导：

$$\begin{aligned} c_t &= \partial_{z_{good}} z_{A_t} - \partial_{z_{good}} \log Z(s) \\ &= I[A_t = good] - \pi(good | s) \end{aligned}$$

$I[A_t = good]$ 是 Indicator Function（指示函数）：采样到 good 时等于 1，采样到 bad 时等于 0。第一项来自被选动作自己的 logit，第二项来自 Softmax 分母的归一化作用。

当前两个动作的概率都是 0.5，因此：

$$A_t=good: c_t=1-0.5=0.5$$

$$A_t=bad: c_t=0-0.5=-0.5$$

- 采样到较好动作时，增大 z_{good} 会提高该动作的概率，所以梯度为正。
- 采样到较差动作时，增大 z_{good} 会挤压较差动作的概率，所以已采样动作的 log probability 对它的梯度为负。

7.6.2.2 不使用 Baseline 时的样本梯度

设较好动作回报为 -80，较差动作回报为 -120。样本梯度分量等于 $G_t c_t$ ：

$$A_t=good: G_t c_t=(-80) \times 0.5=-40$$

$$A_t=bad: G_t c_t=(-120) \times (-0.5)=60$$

两个动作各以 0.5 概率出现，因此平均梯度为：

$$0.5 \times (-40) + 0.5 \times 60 = 10$$

平均梯度为正，最终仍会提高较好动作的概率；但单次采样到较好动作时却得到 -40，会暂时沿相反方向更新。

7.6.2.3 使用 Baseline 后的样本梯度

取状态 baseline 为 -100，两个动作的相对回报分别为：

$$good: -80 - (-100) = 20$$

$$bad: -120 - (-100) = -20$$

新的样本梯度分量为：

$$A_t=good: 20 \times 0.5 = 10$$

$$A_t=bad: (-20) \times (-0.5) = 10$$

平均梯度仍是 10，但这次两种样本都给出 10。因此 baseline 没改变平均方向，只让单次样本更接近这个平均方向。

7.6.2.4 方差降低具体指什么

不使用 baseline 时，样本梯度可能是 -40 或 60；使用 baseline 后，本例中都是 10。Variance（方差）衡量的正是样本围绕平均值的波动大小。

实际问题不会像这个简化例子一样把方差降到 0，但好的 baseline 通常能减少这种波动，使有限 batch 的平均梯度更可靠。baseline 选择很差时则未必降方差。

7.6.3 Action-independent Baseline 为什么不改变期望梯度

对固定状态 s ，如果 baseline $b(s)$ 不依赖当前动作，则：

$$\begin{aligned} & \mathbb{E}_{A \sim \pi_\theta(\cdot|s)} [\nabla_\theta \log \pi_\theta(A|s) b(s)] \\ &= b(s) \sum_a \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) \\ &= b(s) \sum_a \nabla_\theta \pi_\theta(a|s) \\ &= b(s) \nabla_\theta \sum_a \pi_\theta(a|s) \\ &= b(s) \nabla_\theta 1 = 0 \end{aligned}$$

连续动作时将求和换成积分，结论不变。因此：

$$\mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - b(S_t))]$$

与不减 baseline 的梯度期望相同。

关键限制是：baseline 在给定 S_t 后不能再依赖当前 A_t 。若随意减去 $b(S_t, A_t)$ ，上述归一化证明不再成立，可能改变期望梯度。

7.6.4 为什么学习 $V_{\pi}(s)$ 作为 Baseline

对某个状态，当前策略的平均动作价值为：

$$V_{\pi}(s) = \mathbb{E}_{A \sim \pi_{\theta}(\cdot | s)} [Q_{\pi}(s, A)]$$

它正好表示“在该状态下，按当前策略正常行动能得到多少回报”。因此 $V_{\pi}(s)$ 是判断一个已选动作是否高于当前策略平均水平的自然参考值。

真实 V_{π} 通常不可知，可以训练 $b_{\phi}(s)$ 拟合 Monte Carlo return：

$$L_{baseline}(\phi) = 1/2 \sum_{t=0}^{T-1} [(b_{\phi}(S_t) - G_t)^2]$$

因为：

$$\mathbb{E}[G_t | S_t = s] = V_{\pi}(s)$$

在足够数据和足够表达能力下，回归 G_t 的 baseline 会逼近 $V_{\pi}(s)$ 。

7.6.5 REINFORCE with Baseline 的两个 Loss 与训练流程

Monte Carlo advantage estimate 定义为：

$$\hat{A}_t^{MC} = G_t - b_{\phi}(S_t)$$

policy loss 改为：

$$L_{policy} = - \sum_{t=0}^{T-1} \gamma^t \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{MC}$$

Baseline 仍使用上一节定义的 $L_{baseline}$ 回归完整 G_t 。

两个优化目标的梯度边界必须清楚：

- 更新 policy 时， $\hat{A}_t^{MC}$ 作为已计算的样本权重，不通过它更新 baseline。
- 更新 baseline 时，用 G_t 做回归标签，不通过 G_t 或 policy 反传。

实现中的 `stop_gradient(x)` 是 Stop Gradient (停止梯度)：前向使用 x 的数值，反向不沿 x 继续传播梯度。完整流程为：

```
sample complete episodes with current pi_theta
compute G_t for every time step
baseline prediction = b_phi(s_t)
MC advantage estimate = G_t - stop_gradient(b_phi(s_t))
update theta with policy loss
update phi with regression loss (b_phi(s_t) - G_t)^2
```

因此在计算 policy loss 时，baseline 预测只是样本权重的一部分，不会被 policy loss 更新；参数 ϕ 只由 $L_{baseline}$ 更新。

7.6.6 REINFORCE with Baseline 是不是 Actor-Critic?

结论取决于 baseline 是否学习价值，以及采用哪一种术语口径。

- 如果 $b(S_t)$ 是常数、滑动平均或其他不可学习参考值，它没有 Critic，不能称为 Actor-Critic。
- 如果 $b_\phi(S_t)$ 是学习 $V_\pi(S_t)$ 的价值网络，那么结构上已经有 Actor 和 Learned Value Critic。按宽泛定义，它可以叫 **Monte Carlo Actor-Critic (蒙特卡洛 Actor-Critic)**。
- 但它仍然不是第 8 章重点讲的 **TD Actor-Critic (时序差分 Actor-Critic)**。本课程保留 REINFORCE with Baseline 这个名称，用来突出二者训练信号的区别。

对比项	REINFORCE with Learned Baseline	典型 TD Actor-Critic
Actor 的权重	完整回报 $G_t - b_\phi(S_t)$	由 bootstrap 构造的 TD 或 n-step advantage estimate。
Value 网络标签	完整 Monte Carlo return G_t	奖励加下一状态价值预测。
是否 Bootstrap	否	是。
是否必须等完整未来	通常是	不必，可以用短 rollout 更新。

所以最准确的面试表达是：

REINFORCE with learned state-value baseline 在结构上可以视为 Monte Carlo Actor-Critic，但它不使用 bootstrap；通常所说的标准 Actor-Critic 会让 Critic 使用 TD 或 n-step target，并把相应 advantage estimate 提供给 Actor。

7.6.7 Baseline 能做什么、不能做什么

- action-independent baseline 保证的是“不改变期望梯度”，不是“任意 baseline 都必然降方差”。
- baseline 需要与 G_t 的状态依赖变化足够相关，否则降方差效果可能很弱。
- baseline 预测很差时，甚至可能增加样本方差。
- baseline 只做中心化，不消除环境随机性和未来动作采样带来的所有噪声。

7.7 第五步：Advantage 解释 Baseline 后的学习信号

7.7.1 用 $Q_\pi(s,a) - V_\pi(s)$ 定义理论 Advantage

上一节已经用 $G_t - b_\phi(S_t)$ 表示“本次回报比状态参考水平高多少”。如果 baseline 恰好等于真实状态价值 $V_\pi(s)$ ，固定 (s,a) 后对未来随机性取平均：

$$\begin{aligned} & \mathbb{E}[G_t - V_\pi(s) \mid S_t=s, A_t=a] \\ &= Q_\pi(s,a) - V_\pi(s) \end{aligned}$$

这个条件平均被定义为 Advantage Function (优势函数)：

$$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s)$$

因此 Advantage 不是突然出现的新算法，而是给“动作价值超过状态平均水平的部分”起的名字：

- $V_\pi(s)$ ：当前策略在状态 s 下的平均水平。
- $A_\pi(s,a)$ ：动作 a 相对这个平均水平好多少。

在当前策略的动作分布下，所有动作的 advantage 加权平均为 0：

$$\mathbb{E}_{A \sim \pi(\cdot|s)} [A_\pi(s,A)] = 0$$

7.7.2 理论 $A_\pi(s,a)$ 与样本 $\hat{A}_t^{MC}$ 不是同一个量

理论 Advantage 和训练时拿到的样本权重必须区分：

对象	是什么	如何得到
$A_\pi(s,a)$	固定 (s,a) 后的理论条件期望	$Q_\pi(s,a) - V_\pi(s)$ 。
$\hat{A}_t^{MC}$	一条轨迹给出的带噪声估计	$G_t - b_\phi(s_t)$ 。

若 b_ϕ 很好地近似 V_π ，则固定 (s,a) 反复采样时：

$$\mathbb{E} [\hat{A}_t^{MC} | S_t=s, A_t=a] \approx A_\pi(s,a)$$

7.7.3 数值例子：条件期望 2.8 与单次估计 6

某状态有三个动作：

动作	$\pi(a s)$	$Q_\pi(s,a)$
left	0.5	12
stay	0.3	8
right	0.2	4

状态价值为：

$$V_\pi(s) = 0.5 \times 12 + 0.3 \times 8 + 0.2 \times 4 = 9.2$$

所以理论上，left 的 advantage 为：

$$A_\pi(s, \text{left}) = 12 - 9.2 = 2.8$$

但某次实际采样 left 时，可能观测到 $G_t=15$ ，而 baseline 预测 $b_\phi(s)=9$ ：

$$\hat{A}_t^{MC} = 15 - 9 = 6$$

6 是单条轨迹的估计，不等于理论值 2.8；在相同 (s,a) 下反复采样后，它的平均值才会接近理论 advantage。本章到这里仍然只使用完整 G_t ，其他 advantage 估计方法放到第 8 章引入后再比较。

7.8 REINFORCE 系列的统一视图：目标、梯度估计与训练 Loss

7.8.1 REINFORCE with Baseline 的 30 秒推导主线

Policy Gradient 用 log-derivative trick 得到 $\nabla \log \pi$ 形式；REINFORCE 用完整轨迹的 G_t 做 Monte Carlo 估计；因为该估计方差高，再减去 action-independent baseline，并用 $G_t - b_\phi(s_t)$ 估计 advantage。

7.8.2 四个对象： J 、 ∇J 、 g_{MC} 与 L_{surr}

对象	数学类型	是否能直接从有限数据精确得到	作用
$J(\theta)$	标量期望目标	不能	真正希望最大化的期望回报。
$\nabla_{\theta} J(\theta)$	期望梯度向量	不能	理论上的上升方向。
$g_{MC}(\tau)$	随机梯度向量	能，由采样轨迹估计	近似 $\nabla_{\theta} J$ 。
$L_{surv}(\theta; \tau)$	标量代理 loss	能	使 $\nabla L_{surv} = -g_{MC}$ ，方便梯度下降实现。

因此训练代码虽然调用 `loss.backward()`，算法本质仍是用轨迹样本估计 Policy Gradient。代理 loss 是接口，不是新的优化目标。

7.8.3 五个价值量： G 、 Q 、 V 、 A 与 $\hat{A}^{MC}$

符号	层次	是样本还是期望	用途
G_t	单条轨迹	样本	REINFORCE 直接使用的 Monte Carlo return。
$Q_{\pi}(s, a)$	固定状态动作	条件期望	表示特定动作的平均未来回报。
$V_{\pi}(s)$	固定状态	条件期望	表示当前策略在该状态的平均水平。
$A_{\pi}(s, a)$	固定状态动作	条件期望	表示动作相对状态平均水平的增益。
$\hat{A}_t^{MC}$	单条轨迹	样本估计	实现中用于 policy loss 的带噪声优势估计。

7.8.4 REINFORCE with Baseline 的端到端训练流程

1. policy network 输出动作分布
2. 从分布采样动作并保存 `log_prob`
3. 环境生成奖励和下一状态
4. episode 结束后反向计算 `G_t`
5. 可选：baseline 从状态预测参考回报
6. 构造权重 `G_t` 或 `G_t - b_phi(s_t)`
7. 用 `-log_prob * weight` 更新 policy
8. 若有 baseline, 用 `(b_phi(s_t) - G_t)^2` 单独更新它

7.9 策略分布的工程实现：离散、连续与混合动作

前面的推导只要求策略能够采样动作，并计算该动作的 log probability。本节说明离散、连续和混合动作如何实现这一统一接口，不会引入新的 Policy Gradient 算法。

7.9.1 统一接口：采样动作并计算其 Log Probability

REINFORCE 的整体 loss 形式不变，变的是 $\log \pi_{\theta}(a_t | s_t)$ 的计算方式：

动作空间	$\log \pi_{\theta}(a_t s_t)$
离散 categorical	被采样类别的 log probability。
多维连续 Gaussian	各维 Gaussian log density 之和。

动作空间	$\log \pi_{\theta}(a_t s_t)$
离散 + 连续混合	离散类型 log probability + 被选中连续分支的 log density。

因此先统一理解“策略输出一个可采样、可计算 log probability 的动作分布”，再看不同动作空间怎样实现这个接口。

7.9.2 策略网络为什么输出条件分布

随机策略不直接输出“唯一正确动作”，而是输出给定状态下的动作分布：

$$A_t \sim \pi_{\theta}(\cdot | S_t)$$

训练时需要保存采样动作的 $\log \pi_{\theta}(A_t | S_t)$ ，因为后面的 Policy Gradient 会用它建立“动作发生的可能性”与“动作之后的回报”之间的联系。

7.9.3 离散动作：Categorical Policy

若动作是有限个互斥选项，例如 left / stay / right，网络先输出 logits：

$$z = f_{\theta}(s)$$

再用 softmax 得到 categorical distribution（类别分布）：

$$\pi_{\theta}(a=i | s) = \exp(z_i) / \sum_k \exp(z_k)$$

策略从这个分布采样一个动作。概率之和自动为 1，因此提高一个动作的概率会相对压低其他动作。

7.9.4 连续动作：Diagonal Gaussian Policy

若动作是 d 维连续向量：

$$a = (a_1, a_2, \dots, a_d)$$

常见做法是让网络为每一维输出均值和标准差：

$$\mu_{\theta}(s), \sigma_{\theta}(s) > 0$$

若各维在给定状态后条件独立，联合概率密度分解为：

$$\pi_{\theta}(a | s) = \prod_{j=1}^d N(a_j; \mu_{\theta_j}(s), \sigma_{\theta_j}(s))$$

两边取对数：

$$\log \pi_{\theta}(a | s) = \sum_{j=1}^d \log N(a_j; \mu_{\theta_j}(s), \sigma_{\theta_j}(s))$$

符号逐项解释：

- N ：花体大写 N ，表示 Normal Distribution（正态分布，也叫高斯分布）的概率密度函数。
- a_j ：本次实际采样的第 j 维动作值。
- $\mu_{\theta_j}(s)$ ：策略网络预测的第 j 维均值。
- $\sigma_{\theta_j}(s)$ ：策略网络预测的第 j 维标准差；实现中常输出 `log_std` 再取指数，以保证标准差为正。
- 分号 `;`：左边是“在哪个动作值处计算密度”，右边是高斯分布的参数。
- $\prod_j$ 变成 $\sum_j \log$ ：条件独立时联合密度等于各维密度的乘积，对数把乘积变成求和。

例如二维动作为“转向、油门”：

$$a=(0.2,0.7), \mu_{\theta}(s)=(0.1,0.6), \sigma_{\theta}(s)=(0.2,0.1)$$

这次动作的 joint log density (联合对数密度) 是：

$$\log \pi_{\theta}(a/s) = \log N(0.2; 0.1, 0.2) + \log N(0.7; 0.6, 0.1)$$

连续变量取到某个精确数值的概率为 0，因此这里严格来说是 probability density (概率密度)，不是离散动作的 probability mass (概率质量)。工程中通常都简称 `log_prob`。

7.9.5 离散加连续：Hybrid Action Policy

混合动作同时包含离散类型和连续参数：

$$A_t = (K_t, U_t)$$

例如：

游戏：K_t = 选择技能，U_t = [瞄准角度，释放距离]
交易：K_t = {不操作，买入，卖出}，U_t = [价格，数量]

若连续参数的语义由离散类型决定，应使用 conditional factorization (条件分解)：

$$\pi_{\theta}(k, u/s) = \pi_{\theta_d}(k/s) \pi_{\theta_c}(u/s, k)$$

所以联合 log probability 为：

$$\log \pi_{\theta}(k, u/s) = \log \pi_{\theta_d}(k/s) + \log \pi_{\theta_c}(u/s, k)$$

一次采样的顺序是：

```
state s
-> categorical head 采样离散类型 k
-> 选中 k 对应的 continuous head
-> 从该连续分布采样 u
-> 环境执行联合动作 (k, u)
```

如果每个离散类型都有自己的连续参数分支，只有本次被选中的分支会通过 $\log \pi_{\theta_c}(u/s, k)$ 获得直接梯度。未选中分支没有对应的实际动作和回报，不应把它们的 log probability 都加进 loss。

7.9.6 有界连续动作

真实动作常有范围约束，例如方向盘角度必须在 $[-l, l]$ 。直接从无界 Gaussian 采样后用 `clip` 截断，会改变实际执行分布，使得记录的 log probability 与真实动作不一致。

常见处理是：

1. 从 Gaussian 采样无界变量。
2. 用 `tanh` 压缩到 $[-l, l]$ 。
3. 缩放到环境的合法范围。
4. 计算 log probability 时加入 change of variables (变量变换) 带来的 Jacobian correction (雅可比校正)。

本章不推导该校正公式，但要记住原则：动作经过可微变换后，log probability 必须对应变换后的真实分布。

下面用三个短例子确认：同一个样本权重怎样作用到不同策略分布。

本节不引入新的策略类型，只把第 7.9 节定义的 Categorical、Gaussian 和 Hybrid Policy 代入同一个样本 loss：

$$L_t = -\hat{A}_t^{MC} \log \pi_\theta(a_t | s_t)$$

7.9.7 Categorical Policy 的一次更新例子

第 7.9.3 节已经定义：网络输出 logits，经 softmax 归一化后得到 Categorical Policy 的动作概率。有些资料简称它为 Softmax Policy，但 softmax 只是把 logits 转成概率的函数，不是另一种强化学习算法。

某状态下两个动作的当前概率为：

动作	当前概率
left	0.4
right	0.6

本次采样到 left，且 $G_t=10$ 。

- 无 baseline 时，最小化 $-\log \pi(\text{left}|s) \times 10$ 会提高 left 的概率。
- 若 baseline 为 7，则 $\hat{A}_t^{MC}=3$ ，仍提高 left 的概率，但更新强度取决于它高出参考水平多少。
- 若 baseline 为 12，则 $\hat{A}_t^{MC}=-2$ ，说明它虽然获得正回报，却低于该状态的预期水平，因此会降低 left 的概率。

7.9.8 Gaussian Policy 的一次更新例子

假设单维动作是方向盘角度，策略输出：

$$\mu_\theta(s)=0.1, \sigma_\theta(s)=0.2$$

本次采样 $a_t=0.3$ ，并得到正的 $\hat{A}_t^{MC}$ 。最小化：

$$-\log N(0.3; 0.1, 0.2) \hat{A}_t^{MC}$$

会提高在该状态下采样到 0.3 附近动作的密度。梯度可能同时调整均值和标准差，不是简单把均值直接改成 0.3。

7.9.9 Hybrid Policy 的一次更新例子

假设策略先采样：

```
k_t = 火球术
u_t = [角度 20°, 距离 8]
```

训练信号为：

$$-\hat{A}_t^{MC} [\log \pi_\theta(\text{fireball} | s_t) + \log \pi_\theta([20, 8] | s_t, \text{fireball})]$$

若 $\hat{A}_t^{MC} > 0$ ：

- 离散分支会提高相似状态下选择火球术的概率。
- 火球术对应的连续分支会提高采样类似角度和距离的密度。
- 其他未选中技能的连续参数分支不获得这次动作的直接梯度。

7.10 本章面试高频问题

7.10.1 Policy Gradient 为什么使用 $\nabla \log \pi$?

Log-derivative Trick 将 $\nabla p_\theta(\tau)$ 改写为 $p_\theta(\tau) \nabla \log p_\theta(\tau)$, 使梯度可以写成当前轨迹分布下的样本期望。展开轨迹 log probability 后, 只剩各时刻的 $\nabla \log \pi_\theta(A_t | S_t)$ 。

7.10.2 样本梯度与代理 Loss 是什么关系?

$g_{MC}(\tau)$ 是一条轨迹给出的 Policy Gradient 样本估计; L_{surv} 是为自动微分构造的标量。因为 $\nabla_\theta L_{surv} = -g_{MC}$, 最小化代理 loss 与沿样本梯度上升完全等价。它们不是两个连续算法步骤, 也没有两套学习信号。

7.10.3 G_t 和 $Q_\pi(s, a)$ 是什么关系?

G_t 是一条具体轨迹上的随机回报; $Q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$ 是固定 (s, a) 后对所有可能未来取平均。REINFORCE 用实际采样的 G_t 作为 $Q_\pi(s, a)$ 的 Monte Carlo 样本。

7.10.4 Reward-to-go 在推导中的作用是什么?

它将当前动作无法影响的过去奖励从当前梯度权重中去掉, 保留当前动作之后的奖励。这是基于因果关系的 credit assignment, 可以在不改变梯度期望的前提下降低方差。

7.10.5 Baseline 为什么不改变期望梯度?

给定状态后, 若 baseline 不依赖当前动作, 则 $\mathbb{E}_{A \sim \pi}[\nabla \log \pi(A|s)b(s)] = 0$ 。因此减去 baseline 只会改变样本梯度的中心和方差, 不改变期望。

7.10.6 为什么常用 $V_\pi(s)$ 作为 Baseline?

$V_\pi(s)$ 是在状态 s 下按当前策略选择动作的平均未来回报, 正好是判断已选动作相对当前策略平均水平好坏的参考点。它未知时, 可用 $b_\phi(s)$ 回归 Monte Carlo return G_t 。

7.10.7 理论 Advantage 和 $G_t - b_\phi(s_t)$ 是同一个东西吗?

不是。 $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$ 是理论条件期望; $G_t - b_\phi(s_t)$ 是一条轨迹上的 Monte Carlo 估计。当 baseline 近似 V_π 时, 后者的条件期望会近似前者。

7.10.8 Softmax 的 Score Function Gradient 为什么是 $1[A_t=i] - \pi(i|s)$?

因为 $\log \pi(A_t|s) = z_{A_t} - \log \sum_j \exp(z_j)$ 。对动作 i 的 logit z_i 求偏导时, 第一项在 $A_t=i$ 时贡献 1、否则贡献 0; 归一化项恒贡献 $\pi(i|s)$, 所以结果为 $1[A_t=i] - \pi(i|s)$ 。

7.10.9 REINFORCE with Baseline 是不是 Actor-Critic?

固定或不可学习 baseline 没有 Critic。若 baseline 是学习 $V_\pi(s)$ 的网络, 宽泛地可以称为 Monte Carlo Actor-Critic; 但它仍用完整 G_t 训练, 不 bootstrap。典型 TD Actor-Critic 则用 reward 加下一状态价值构造 target, 本课程据此把两者分在第 7、8 章。

7.11 本章练习

1. 从 $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[G_0]$ 推导到 trajectory-level Policy Gradient。
2. 展开 $p_\theta(\tau)$, 说明环境转移项为什么不需要求导, 以及环境为什么仍然影响梯度样本。
3. 说明从 G_0 换成 reward-to-go G_t 的因果逻辑。
4. 使用 iterated expectation 说明 $Q_\pi(s, a)$ 如何从 G_t 的条件期望中出现。
5. 证明 action-independent baseline 不改变 Policy Gradient 的期望。
6. 说明为什么 $V_\pi(s)$ 是自然 baseline, 以及真实 V_π 未知时如何训练近似。
7. 对比 $A_\pi(s, a)$ 与 $\hat{A}_t^{MC}$ 。
8. 写出二维 diagonal Gaussian policy 的 joint log probability。
9. 对混合动作 $A_t = (K_p U_p)$, 写出 conditional policy factorization 和对应 REINFORCE loss。
10. 说明为什么 Categorical、Gaussian 和 Hybrid Policy 可以共用同一个 REINFORCE loss 结构。

11. 区分 $J(\theta)$ 、 $g_{MC}(\tau)$ 和 $L_{surv}(\theta; \tau)$, 并说明梯度下降为何等价于样本梯度上升。
12. 从两个动作的 Softmax 定义推导 $\partial_{z_{good}} \log \pi(A_t|s) = I[A_t=good] - \pi(good|s)$ 。
13. 解释 learned state-value baseline 在什么意义下可以叫 Critic, 以及它与典型 TD Actor-Critic 的关键区别。

▼ 参考答案 (点击展开)

1. **Trajectory-level 推导**: 先将期望写成积分, 得到 $\nabla J = \int \nabla p_\theta(\tau) G_0 d\tau$; 使用 $\nabla p = p \nabla \log p$ 得到 $\mathbb{E}[\nabla \log p_\theta(\tau) G_0]$ 。
2. **轨迹分解**: $p_\theta(\tau) = p(S_0) \prod_t \pi_\theta(A_t|S_t) P(S_{t+1}|S_t, A_t)$ 。 $p(S_0)$ 和 P 不依赖 θ , 所以其 log-gradient 为 0; 但环境转移仍决定哪些轨迹、状态和奖励会被采样, 因此影响梯度样本的分布。
3. **Reward-to-go**: 时刻 t 的动作无法影响过去奖励, 将过去奖励乘到当前 score function 上只会带来噪声。保留动作之后的 G_t 能进行时间 credit assignment。
4. **Q_π 的来源**: 固定 (S_t, A_t) 后, policy score 已是确定量。根据迭代期望, $\mathbb{E}[\nabla \log \pi G_t] = \mathbb{E}[\nabla \log \pi \mathbb{E}[G_t|S_t, A_t]]$, 而内层期望定义为 $Q_\pi(S_t, A_t)$ 。
5. **Baseline 证明**: 给定 s , $b(s) \sum_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) = b(s) \nabla \sum_a \pi_\theta(a|s) = b(s) \nabla 1 = 0$ 。
6. **V_π Baseline**: $V_\pi(s) = \mathbb{E}_{A \sim \pi}[Q_\pi(s, A)]$ 是该状态下当前策略的平均回报水平。未知时可训练 $b_\phi(s)$ 最小化 $(b_\phi(s_t) - G_t)^2$ 。
7. **Advantage 对比**: $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$ 是理论期望量; $\hat{A}_t^{MC} = G_t - b_\phi(s_t)$ 是单条轨迹上的带噪声估计。
8. **二维 Gaussian**: $\log \pi_\theta(a|s) = \log N(a_1; \mu_{\theta,1}(s), \sigma_{\theta,1}(s)) + \log N(a_2; \mu_{\theta,2}(s), \sigma_{\theta,2}(s))$ 。
9. **混合动作**: $\pi_\theta(k, u|s) = \pi_{\theta_d}(k|s) \pi_{\theta_c}(u|s, k)$; 样本 loss 为 $-\gamma' G_t [\log \pi_{\theta_d}(k_t|s_t) + \log \pi_{\theta_c}(u_t|s_t, k_t)]$ 。
10. **统一 Loss**: 三类策略都能计算已采样动作的 $\log \pi_\theta(a_t|s_t)$, 因此都可使用 $-\hat{A}_t^{MC} \log \pi_\theta(a_t|s_t)$ 。区别只在 log probability 的计算方式: 离散动作取类别概率, 多维连续动作累加各维 log density, 混合动作累加离散分支与已选连续分支的 log probability。
11. **三个优化对象**: $J(\theta)$ 是真正希望最大化、但无法精确计算的期望回报; $g_{MC}(\tau)$ 是一条轨迹对 ∇J 的随机估计; L_{surv} 是自动微分需要的标量包装。由于 $\nabla L_{surv} = -g_{MC}$, 梯度下降 $\theta \leftarrow \theta - \alpha \nabla L_{surv}$ 等于样本梯度上升 $\theta \leftarrow \theta + \alpha g_{MC}$ 。
12. **Softmax Score Gradient**: 令 $Z = \sum_j \exp(z_j)$, 则 $\log \pi(A_t|s) = z_{A_t} - \log Z$ 。对 z_{good} 求偏导, 前一项得到 $I[A_t=good]$, 后一项得到 $\exp(z_{good}) Z^{-1} = \pi(good|s)$, 两者相减即得结论。
13. **与 Actor-Critic 的边界**: 不可学习 baseline 不是 Critic。学习 $V_\pi(s)$ 的 baseline 在结构上可以视为 Monte Carlo Critic, 但 REINFORCE with Baseline 仍用完整 G_t 作为价值标签和 Actor 权重, 不使用 bootstrap; 典型 TD Actor-Critic 用 reward 加下一状态价值构造 TD 或 n-step target, 可以在短 rollout 后更新。